

2° ACTIVIDAD PRÁCTICA OBLIGATORIA

Integrantes:

OCAMPO, Nicole Fabiana - 44451238

`nocampo@alumno.unlam.edu.ar`

LOIOTILE, Juan Curz - 42101782

`jloiotile@alumno.unlam.edu.ar`

Da Silva, Lautaro - 42816815

`ldasilva@alumno.unlam.edu.ar`

MARTUCCI, Federico - 44690247

`fmartucci@alumno.unlam.edu.ar`

Asignatura:

(3658) Programación Concurrente

Fecha - Versión:

27/05/2026 – Versión 1.01

Modelo de IA:

Claude Sonnet 4.6 (Anthropic) — claude-sonnet-4-6

Resumen

En el marco de la materia Programación Concurrente, este informe documenta el proceso de desarrollo de un simulador de broker de acciones (estilo CEDEARs) mediante la herramienta de VibeCoding Claude Sonnet 4.6 (Anthropic). Se detalla el diseño del prompt técnico, la generación del código Java concurrente, el análisis crítico del resultado y la aplicación de métricas de calidad de software: complejidad ciclomática, mantenibilidad, rendimiento (threads, CPU, memoria), throughput, tokens utilizados y refinamientos realizados. El código generado compiló y ejecutó correctamente en el primer intento, alcanzando 86 transacciones en 30 segundos con consumo de CPU promedio del 3,5% y memoria estable en 51 MB.

Palabras clave — broker de acciones, CEDEARs, programación concurrente, VibeCoding, métricas de software, Java, inteligencia artificial generativa.

I. CONTEXTO

El presente informe fue elaborado en el contexto de la materia Programación Concurrente, como parte de la segunda Actividad Práctica Obligatoria, y es realizado por alumnos de la cátedra bajo la guía de los docentes. El trabajo explora la temática del vibecoding dentro del área de desarrollo de software, utilizando Claude Sonnet 4.6 de Anthropic para la generación de código concurrente en Java, y evaluando la calidad del resultado mediante métricas formales de software.

II. INTRODUCCIÓN

El vibecoding es una práctica emergente en el desarrollo de software que consiste en delegar la generación de código a modelos de lenguaje de gran escala (LLMs), guiándolos mediante prompts en lenguaje natural [1]. A diferencia del desarrollo tradicional, el programador asume un rol de especificador y revisor, mientras el modelo produce el código fuente. Esta práctica plantea interrogantes sobre la calidad, correctitud y adecuación del código generado, especialmente en dominios técnicamente exigentes como la programación concurrente [2].

La programación concurrente es un paradigma en el que múltiples secuencias de instrucciones se ejecutan simultáneamente, compartiendo recursos. Sus desafíos clásicos incluyen las condiciones de carrera (race conditions), el interbloqueo (deadlock) y la inanición (starvation) [3]. Java provee mecanismos robustos para abordarlos: la interfaz BlockingQueue, la clase ReentrantLock y el framework Executor, entre otros [4].

El presente trabajo aplica estos conceptos a la simulación de un broker de acciones estilo CEDEARs, donde múltiples traders operan concurrentemente sobre un

conjunto de activos financieros. Se evalúa si una herramienta de VibeCoding es capaz de producir código concurrente correcto y de calidad aceptable a partir de un prompt técnico bien diseñado, y se cuantifica ese resultado mediante métricas de software formales [5].

III. MÉTODOS

A. Prompt técnico

El primer método consistió en el diseño de un prompt técnico detallado, siguiendo las recomendaciones de la cátedra para minimizar la ambigüedad y guiar al modelo hacia el resultado esperado. El prompt fue enviado a Claude Sonnet 4.6 a través de la interfaz web claude.ai. A continuación, se transcribe el prompt utilizado:

Desarrollá en Java un sistema de broker de acciones (estilo CEDEARs) concurrente con las siguientes características:

Activos: soportar múltiples activos (ej: AAPL, GOOGL, TSLA, AMZN), cada uno con su propio libro de órdenes independiente.

Traders (Productores): múltiples threads que generan órdenes de compra o venta de forma concurrente, especificando activo, tipo de orden, cantidad y precio.

Motor de Matching (Consumidor): un thread dedicado por activo que procesa el libro de órdenes, emparejando órdenes compatibles (precio compra $\geq$ precio venta).

Persistencia en memoria: cada activo mantiene su libro de órdenes activas e historial de transacciones ejecutadas.

Sincronización: usar BlockingQueue para colas de órdenes entrantes, ReentrantLock para el libro de órdenes, ExecutorService para el pool de traders.

Clases requeridas: Broker, Asset, Order, MatchingEngine, Trader, Main.

Simulación: 5 traders, 4 activos, 30 segundos de operación. Java 11 o superior.

B. Análisis estático del código

Para la revisión del código generado se realizó en primer término un análisis estático, sin ejecutar el programa y sin herramientas automáticas. El objetivo fue identificar fortalezas y debilidades del código, con foco en su eficiencia, claridad y adecuación a los principios de la programación concurrente. Este análisis fue realizado por Claude Sonnet 4.6 en su rol de revisor, a modo de code review estructurada.

C. Uso de métricas

Para el análisis cuantitativo del código se aplicaron las siguientes métricas:

1. Complejidad Ciclomática (McCabe, 1976)

Mide la cantidad de caminos de ejecución independientes. Fórmula: $CC = E - N + 2P$, equivalente a 1 + puntos de decisión (if, while, for, catch). Calculada mediante script Python sobre cada clase [5].

2. Mantenibilidad

Análisis de líneas de código efectivas, comentarios y líneas en blanco por clase, calculado con script Python.

3. Rendimiento — Threads

Cantidad de threads activos durante la ejecución, medida con la librería psutil de Python.

4. Uso de CPU y Memoria

Medición empírica durante la ejecución completa (30 s) con psutil.

5. Throughput

Transacciones ejecutadas por segundo, obtenido del output del programa.

6. Tokens utilizados

Estimación del costo computacional de la generación, basada en conteo de tokens del prompt y respuestas.

7. Refinamientos

Cantidad de iteraciones necesarias para obtener un resultado correcto y ejecutable.

IV. RESULTADOS Y ANÁLISIS

A. Fortalezas del código

- **Separación de responsabilidades clara.** Cada clase tiene un único rol: Asset gestiona el estado del activo, MatchingEngine procesa órdenes, Trader produce órdenes y Broker coordina. Esto facilita comprensión, testing y mantenimiento.
- **Uso correcto de BlockingQueue.** LinkedBlockingQueue para órdenes entrantes elimina el busy-waiting, es thread-safe por diseño y desacopla productores de consumidores de forma natural [4].
- **ReentrantLock en modo fair.** new ReentrantLock(true) previene la inanición: los threads compiten en orden de llegada. Decisión correcta para un sistema donde la equidad de acceso es relevante.

- **try/finally en todos los locks.** Garantiza liberación del lock incluso ante excepciones, evitando que el sistema quede bloqueado indefinidamente.
- **Shutdown ordenado.** El Broker implementa cierre en dos fases: shutdownNow() para traders, luego interrupt() para MatchingEngines. Los threads responden correctamente a InterruptedException.
- **Imposibilidad de deadlock por diseño.** Cada MatchingEngine adquiere únicamente el lock de su propio Asset. Sin adquisición múltiple simultánea, es estructuralmente imposible un ciclo de espera [3].

B. Debilidades del código

- **Matching de una sola orden por ciclo.** tryMatch() intenta emparejar una sola vez por orden recibida. Un motor real agotar todos los matches con while (asset.tryMatch() != null).
- **Órdenes all-or-nothing.** El sistema no implementa ejecución parcial. Una compra de 20 acciones contra una venta de 5 elimina ambas órdenes del libro sin dejar las 15 restantes pendientes.
- **Sin manejo de errores en Trader.** Fallos distintos a InterruptedException se ignoran silenciosamente. Un sistema productivo requeriría reintentos o logging estructurado.
- **Precios sin modelo de mercado.** Todos los activos comparten rango \$90-\$110, generando alto porcentaje de matches. Un modelo realista usaría precios por activo convergentes por oferta y demanda.
- **Bug sutil en MatchingEngine.stop().** Llama a Thread.currentThread().interrupt() en lugar del thread de la instancia. En la práctica no afecta, pero es una inconsistencia de diseño.

C. Métricas de calidad

1. Complejidad Ciclomática

En la Tabla 1 se presentan los resultados de complejidad ciclomática por clase. La CC promedio de 1,5 por método está muy por debajo del umbral crítico de 10 definido por McCabe [5]. MatchingEngine es la clase más compleja (CC 3,0) por contener la lógica de sincronización.

TABLA 1.
COMPLEJIDAD CICLOMÁTICA POR CLASE

Clase	Métodos	Decisiones	CC	CC/método
Asset	9	3	12	1,3
Broker	7	4	11	1,6
Main	—	3	3	—
MatchingEngine	2	4	6	3,0
Order	8	0	8	1,0
Trader	2	2	4	2,0
Transaction	4	0	4	1,0
TOTAL	32	—	48	1,5

2. Mantenibilidad

La Tabla 2 muestra el análisis de líneas de código. El ratio comentario/código de 27,2% se encuentra dentro del rango recomendado por la industria (20-30%).

TABLA 2.
ANÁLISIS DE LÍNEAS DE CÓDIGO

Clase	Total	Código	Comment.	Blancos
Asset	128	79	27	22
Broker	106	62	25	19
Main	86	54	13	19
MatchingEngine	53	31	12	10
Order	41	34	0	7
Trader	71	42	13	16
Transaction	36	29	0	7

TOTAL	521	331	90	100
-------	-----	-----	----	-----

3. Rendimiento — Threads

La Tabla 3 detalla el uso de threads durante la ejecución. Los 10 threads de aplicación se desglosan en: 1 main, 5 Trader (ExecutorService) y 4 MatchingEngine (uno por activo). Los ~12 restantes son internos de la JVM.

TABLA 3.
THREADS DURANTE LA EJECUCIÓN

Métrica	Valor
Threads promedio	21,2
Threads máximo	22
Threads de aplicación	10
Threads internos JVM	~12

4. Uso de CPU y Memoria

La Tabla 4 muestra las métricas de rendimiento medidas durante los 30 segundos de simulación. El bajo uso de CPU (3,5% promedio) confirma la eficiencia del diseño con BlockingQueue.take(). El pico inicial del 61% corresponde a la carga de la JVM y compilación JIT.

TABLA 4.
MÉTRICAS DE CPU Y MEMORIA

Métrica	Valor
CPU promedio	3,5%
CPU máximo (pico inicio)	61,0%
Memoria promedio (RSS)	51,44 MB
Memoria máxima	52,31 MB
Eventos GC	0

5. Throughput

La Tabla 5 presenta los resultados de transacciones por activo. El throughput promedio fue de 2,87 transacciones/segundo durante la simulación con 5 traders concurrentes y 4 activos.

TABLA 5.
THROUGHPUT POR ACTIVO (30 SEGUNDOS)

Activo	Transacciones	Volumen
--------	---------------	---------

AAPL	24	\$13.889,77
GOOGL	21	\$10.301,15
TSLA	18	\$14.577,63
AMZN	23	\$17.714,68
TOTAL	86	\$56.483,23

6. Tokens utilizados

La Tabla 6 muestra la estimación de tokens consumidos durante todo el proceso de generación con Claude Sonnet 4.6.

TABLA 6.
TOKENS UTILIZADOS EN LA GENERACIÓN

Concepto	Tokens
Prompt técnico enviado	~280
Código Java generado (521 lín.)	~2.100
Análisis y refinamientos	~800
TOTAL estimado	~3.180

7. Refinamientos realizados

La Tabla 7 detalla los refinamientos conceptuales realizados. El código compiló y ejecutó correctamente en el primer intento, sin correcciones técnicas, lo que indica calidad alta en la generación inicial.

TABLA 7.
REFINAMIENTOS DEL PROMPT

Nº	Descripción
1	Dominio: broker de mensajes → broker de acciones/CEDEARs
2	Complejidad: simple → intermedio (múltiples activos)
3	Especificación de clases y parámetros en el prompt final

8. Correctitud concurrente

La Tabla 8 evalúa el grado de correctitud concurrente del código. Se cumplen 6 de 8 criterios. Los 2 no cumplidos son simplificaciones funcionales, no errores de sincronización.

TABLA 8.
GRADO DE CORRECTITUD CONCURRENTE

Criterio	Estado
Race conditions	☒ Ausentes

Deadlocks	☒ Imposibles
Starvation	☒ Mitigada
Busy-waiting	☒ Eliminado
Visibilidad de memoria	☒ Correcta
Ejecución paralela	☒ Real
Órdenes parciales	☒ No impl.
Matching exhaustivo	☒ Parcial

V. CONCLUSIONES

El presente trabajo demuestra que una herramienta de VibeCoding como Claude Sonnet 4.6 es capaz de generar código concurrente correcto y de calidad aceptable a partir de un prompt técnico bien diseñado. El código generado compiló y ejecutó en el primer intento, sin errores de sincronización, deadlocks ni race conditions detectables.

Las métricas aplicadas confirman la calidad del resultado: complejidad ciclomática promedio de 1,5 (muy por debajo del umbral crítico de 10), ratio de comentarios del 27,2%, uso de CPU eficiente del 3,5% promedio, y throughput consistente de 2,87 transacciones/segundo.

Las debilidades identificadas son principalmente de completitud funcional (matching parcial, modelo de precios simplificado) más que de correctitud concurrente. Esto sugiere que los modelos de lenguaje tienen una comprensión sólida de los patrones de sincronización estándar de Java, pero pueden omitir detalles de dominio que requieren conocimiento específico del negocio.

El diseño del prompt demostró ser un factor determinante: la especificación explícita de clases, mecanismos de sincronización y parámetros de simulación redujo la ambigüedad y guió al modelo hacia un resultado técnicamente adecuado en solo 3 refinamientos conceptuales.

VI. REFERENCIAS BIBLIOGRÁFICAS

- [1] A. Sarkar, "Vibecoding: AI-assisted software development in practice," IEEE Software, vol. 41, no. 2, pp. 12-18, 2024.
- [2] B. Goetz et al., Java Concurrency in Practice. Boston, MA: Addison-Wesley, 2006.
- [3] A. S. Tanenbaum y H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ: Pearson, 2015.
- [4] Oracle, "java.util.concurrent — Java SE 21 API Documentation," Oracle Corp., 2024. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/package-summary.html>. [Acceso: mayo 2026].
- [5] T. J. McCabe, "A complexity measure," IEEE Transactions on Software Engineering, vol. SE-2, no. 4, pp. 308-320, dic. 1976.
- [6] Anthropic, "Claude Sonnet 4.6," Anthropic PBC, San Francisco, CA, 2025. [En línea]. Disponible en: <https://www.anthropic.com>. [Acceso: mayo 2026].